Can Small Language Models Learn Matrix Inversion?

A Three-Level Diagnostic Study of Syntactic, Structural, and Functional Learning via Supervised Fine-Tuning

MatrixInvLLM Experimental Report

May 15, 2026

Abstract

We investigate whether small language models (LLMs) can learn to compute matrix inverses through supervised fine-tuning on synthetic data—a controlled test of whether LLMs can internalize mathematical patterns that are computationally non-trivial for humans to reproduce from memory. Matrix inversion is an ideal probe: it has a verifiable ground truth ($AA^{-1} = I$), requires multi-step algebraic reasoning, and generates output that can look superficially plausible while remaining mathematically wrong.

Using LoRA (Low-Rank Adaptation, rank $r = 16$, $\alpha = 32$) to fine-tune Llama-3.2-3B on 800 supervised (A, A^{-1}) training examples covering 2×2 and 3×3 matrices, we evaluate both the untrained baseline and the fine-tuned model on 100 held-out examples using a multi-dimensional diagnostic suite that distinguishes three levels of success: (1) syntactic well-formedness, (2) coarse numeric resemblance to the true inverse, and (3) exact functional correctness via residual-based metrics.

Our main findings are:

- **Baseline**: 100% of outputs are syntactically parseable and correctly shaped, but 0% satisfy the inverse constraint. All outputs are classified `NUMERICALLY_WRONG`. Average residuals are $\|AX - I\|_F = 1.84$ and $\|XA - I\|_F = 1.85$. The model behaves essentially as a format imitator with no inverse-like numeric content (determinant ratio 0.045, sign agreement 0.503).
- **Fine-tuned**: Fine-tuning on 800 steps improves several coarse numeric diagnostics—determinant ratio rises from 0.045 to 0.639, sign agreement from 0.503 to 0.795, and relative Frobenius error to the gold inverse falls from 1.030 to 0.727—indicating partial internalization of inverse-like numeric structure. However, true inverse accuracy remains 0% under all residual tolerances, and fine-tuning introduces severe generation instability: only 60% of outputs are parseable, with 22% entering repeated-block loops and 18% producing malformed shapes.

The dominant conclusion is that LoRA fine-tuning learned *partial inverse-like numeric structure* but could not bridge the gap to functional correctness, while simultaneously degrading generation reliability. This work provides a controlled diagnostic on the limits of supervised fine-tuning for precise arithmetic computation in small LLMs.

Contents

1 Introduction **3**
- 1.1 Why Matrix Inversion? . . . 3
- 1.2 Research Question and Contribution . . . 3
- 1.3 Key Findings . . . 4
- 1.4 Paper Structure . . . 4

2 Methodology **4**
- 2.1 The Core Scientific Question . . . 4
- 2.2 Matrix Generation and Stability Criteria . . . 4
- 2.3 Data Serialization and Format . . . 5
- 2.4 Train/Val/Test Split . . . 5
- 2.5 Model and Training Setup . . . 5
- 2.6 Evaluation Framework . . . 6
  - 2.6.1 Level 1 — Syntactic Metrics . . . 6
  - 2.6.2 Level 2 — Coarse Numeric Diagnostics . . . 6
  - 2.6.3 Level 3 — True Inverse Correctness . . . 6
  - 2.6.4 Failure Mode Taxonomy . . . 7

3 Experiments and Results **7**
- 3.1 Baseline Behavior (Untrained Model) . . . 7
- 3.2 Fine-Tuned Model Behavior . . . 8
- 3.3 Comparative Summary and Failure Mode Analysis . . . 9
- 3.4 The Three-Level Gap . . . 9

4 Discussion **9**
- 4.1 What the Results Reveal About LLM Arithmetic Reasoning . . . 9
- 4.2 Why the Functional Gap Persists . . . 10
- 4.3 Decoding Instability as a First-Class Finding . . . 10
- 4.4 Implications for the Broader Question . . . 11
- 4.5 Limitations . . . 11

5 Conclusion **11**

A Tables **13**

B Figures **14**

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities across language understanding and generation tasks. However, their ability to perform *precise arithmetic reasoning*—especially on structured numeric problems with verifiable answers—remains poorly understood. A fundamental question is whether supervised fine-tuning can teach a model to internalize a mathematical *function*, or whether it merely teaches the model to produce outputs that *resemble* correct answers in format without satisfying the underlying constraints.

1.1 Why Matrix Inversion?

Matrix inversion is an ideal controlled benchmark for this question. Given a square matrix A, its inverse A^{-1} is the unique matrix satisfying:

$$A \cdot A^{-1} = I \qquad \text{and} \qquad A^{-1} \cdot A = I$$

where I is the identity matrix. Computing this by hand requires multi-step Gaussian elimination, careful row operations, and exact bookkeeping across n^2 entries—an operation that is cognitively demanding and error-prone for humans even at 3×3 scale. Yet the correctness of any candidate inverse X can be verified instantly by checking the *residual norms* $\|AX - I\|_F$ and $\|XA - I\|_F$.

This asymmetry—hard to produce, easy to verify—makes matrix inversion a valuable probe for studying whether LLMs can learn computationally demanding patterns from examples. If a model sees many $(A,\ A^{-1})$ pairs during training, can it distill implicit inverse-computation structure into its weights? Or does it learn only surface regularities, such as the output format or plausible value ranges, without satisfying the algebraic constraint?

1.2 Research Question and Contribution

We investigate: *Can an LLM learn to compute matrix inverses through supervised fine-tuning on synthetic data, and what does the failure mode reveal about LLM arithmetic reasoning?*

To answer this, we design a rigorous experimental framework that:

1. Generates 1,000 stable synthetic matrices (2×2 and 3×3) with determinant and condition number filters to ensure well-posed inverse targets.
2. Uses gold-standard residual validation ($\|AX - I\|_F < 10^{-8}$) rather than symbolic comparison.
3. Employs deterministic text serialization for reproducible parsing and malformation detection.
4. Fine-tunes Llama-3.2-3B using LoRA on 800 training examples for 800 optimization steps.
5. Evaluates using a *three-level diagnostic suite* distinguishing:
    - **Syntactic well-formedness**: Is the output parseable into a valid matrix shape?
    - **Inverse-like numeric structure**: Do coarse diagnostics (determinant ratio, sign agreement, relative Frobenius error) indicate the model has learned something about what inverses look like?
    - **True inverse correctness**: Does the predicted matrix satisfy $\|AX - I\|_F < \varepsilon$ at any tolerance level?

1.3 Key Findings

Our results reveal a layered failure structure. The baseline (untrained) model produces syntactically valid output 100% of the time but with no inverse-like numeric content. Fine-tuning substantially improves coarse diagnostics—the determinant ratio rises from 0.045 to 0.639, sign agreement from 0.503 to 0.795—suggesting that the model has internalized partial inverse-like structure. However, true inverse accuracy remains 0% under all residual-based tolerances, and fine-tuning simultaneously introduces severe generation instability: only 60% of fine-tuned outputs are parseable, with the remainder failing via repetition loops or malformed shapes.

The central finding is therefore not a simple "format learned, function not learned" story, but rather a *graded partial learning* result: fine-tuning moved the model along the spectrum toward the target function without reaching it, and in doing so exposed a decoding reliability bottleneck that complicates interpretation.

1.4 Paper Structure

Section 2 details our data generation, serialization, training setup, and the full evaluation metric suite. Section 3 presents results across all three diagnostic levels. Section 4 interprets the findings, discusses limitations, and outlines targeted future directions.

2 Methodology

2.1 The Core Scientific Question

Our central hypothesis is that matrix inversion is a canonical example of a computation that is *hard for humans to perform from scratch but easy to verify.* A human computing a 3×3 inverse by hand must apply Gaussian elimination across 9 entries over multiple steps—an error-prone, cognitively demanding process. Yet any candidate X can be checked in seconds by computing AX and asking whether it equals I.

This asymmetry motivates a natural test of LLM pattern learning: if a model sees many $(A,\ A^{-1})$ pairs, can it *distill* the inverse-computation pattern into its weights? Equivalently, can supervised fine-tuning substitute for algorithmic computation when the target function is mathematically precise but learnable from examples? We use a controlled synthetic setup to isolate this question from confounds like ambiguous ground truth or data quality issues.

2.2 Matrix Generation and Stability Criteria

We generate synthetic $n \times n$ matrices ($n \in \{2, 3\}$) by uniform random sampling from $[-1, 1]$, followed by stability filtering. To avoid ill-posed or numerically unstable inverse targets, we apply three criteria:

1. **Determinant threshold**: $|\det(A)| > 0.05$ ensures full rank and avoids near-singular matrices whose inverses have extreme entries.
2. **Condition number bound**: $\kappa(A) < 50$ ensures that small perturbations in A do not cause large changes in A^{-1}, making the target well-defined for a generalization study.
3. **Residual validation**: The computed gold inverse X^* must satisfy $\|AX^* - I\|_F < 10^{-8}$, confirming that our ground truth is numerically exact.

We sample up to 20,000 candidates per matrix size and retain 500 examples per size, for a total dataset of 1,000 examples. The experiment covers both 2×2 and 3×3 matrices to probe whether success or failure is size-dependent.

2.3 Data Serialization and Format

Inputs and targets are serialized in a deterministic machine-readable text format. A representative example:

```
MatrixSize: 3
Matrix:
0.39195, -0.18209, -0.65341
-0.68713, -0.49951, 0.09845
0.42919, 0.32039, -0.44013
```

The target output:

```
Inverse:
1.50708, -2.31690, -2.75565
-2.08222, 0.86379, 3.28444
-0.04613, -1.63052, -2.56829
```

Design rationale: (1) Strict formatting ensures reproducible parsing; (2) Row-major, comma-separated ordering makes structural malformations detectable; (3) Fixed 5-decimal precision matches floating-point resolution without excessive token cost.

2.4 Train/Val/Test Split

We use a reproducible 80/10/10 split (seed = 123) across the full 1,000-example dataset:

- **Train**: 800 examples (400 per matrix size)
- **Validation**: 100 examples
- **Test**: 100 examples (48 of size 2×2, 52 of size 3×3)

All reported results are on the held-out test set of 100 examples.

2.5 Model and Training Setup

Base model: Llama-3.2-3B (3 billion parameters; a small but capable open-weights model).

LoRA configuration:

- Rank: $r = 16$
- Alpha: $\alpha = 32$ (effective scaling factor $\alpha/r = 2$)
- Dropout: 0.05

Optimization:

- Optimizer: AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
- Learning rate: 5×10^{-5} with 5% warmup
- Weight decay: 0.01
- Training steps: 800 (with batch size 16)
- Infrastructure: Tinker SDK with async pipelining

Inference: Greedy decoding (temperature = 0.0) with a maximum of 128 new tokens. No hyperparameter search was performed; this is an intentionally minimal configuration.

2.6 Evaluation Framework

2.6.1 Level 1 — Syntactic Metrics

- **Parse success rate**: Percentage of outputs parseable into a valid matrix.
- **Shape correctness**: Among parseable outputs, percentage with correct dimensions ($n \times n$).

2.6.2 Level 2 — Coarse Numeric Diagnostics

The following metrics assess whether a parseable prediction X exhibits inverse-like numeric structure, independent of whether it is exactly correct. All are computed *among parseable outputs only.*

- **Relative Frobenius error**: $\|X - A^{-1}\|_F / \|A^{-1}\|_F$. Values near 0 indicate predictions close to the true inverse; the baseline value near 1.0 indicates predictions as far from the true inverse as the inverse's own magnitude.
- **Determinant ratio**: $\det(X)/\det(A^{-1})$. A perfect inverse has $\det(X) = \det(A^{-1}) = 1/\det(A)$, so a ratio of 1.0 indicates correct determinant scale. A ratio near 0 indicates predictions with negligible determinant relative to the target.
- **Sign agreement**: Fraction of entries where $\text{sign}(X_{ij}) = \text{sign}(A^{-1}_{ij})$. Random output would score near 0.5 for balanced sign distributions; a value approaching 1.0 indicates the model is learning which entries should be positive vs. negative.
- **Spectral error**: Largest singular value of $(X - A^{-1})$. This captures the magnitude of the most adversarial direction of error, complementing the Frobenius norm.
- **Copy-detection distances**:
  - $\|X - A\|_F$: How far the prediction is from simply copying the input.
  - $\|X - A^T\|_F$: How far from the transpose of the input.
  - $\|X - (-A)\|_F$: How far from the negation of the input.

  Large values indicate the model is not trivially copying or negating the input; but these must be read together with the other metrics to interpret the prediction's actual content.

2.6.3 Level 3 — True Inverse Correctness

- **Left residual**: $\|AX - I\|_F$. The primary success criterion.
- **Right residual**: $\|XA - I\|_F$. Ensures commutativity.
- **Tolerance-based accuracy**:
  - **Strict**: $\|AX - I\|_F < 10^{-2}$
  - **Medium**: $\|AX - I\|_F < 5 \times 10^{-2}$
  - **Loose**: $\|AX - I\|_F < 10^{-1}$

  A model that passes even the loose tolerance has demonstrated meaningful inverse computation; all three failing simultaneously means the output is far from satisfying $AX \approx I$.

2.6.4 Failure Mode Taxonomy

We classify each output into one of six categories:

- `EOS_ONLY`: Output contains only end-of-sequence tokens; no matrix produced.
- `PROMPT_ECHO`: Output mirrors the input prompt verbatim.
- `REPEATED_BLOCK_LOOP`: Output enters repetition loops (e.g., the same row repeated until token budget is exhausted).
- `SHAPE_FORMAT`: Output is partially parseable but has incorrect dimensions.
- `NUMERICALLY_WRONG`: Correct shape, but residual $> 10^{-1}$ (loose tolerance failure).
- `SUCCESS`: Residual $< 10^{-2}$ (strict tolerance pass).

This taxonomy distinguishes *decoding failures* (EOS_ONLY, REPEATED_BLOCK_LOOP, SHAPE_FORMAT) from *numeric failures* (NUMERICALLY_WRONG), making it possible to isolate whether the bottleneck is generation reliability or mathematical computation.

3 Experiments and Results

We evaluate both the untrained baseline and the LoRA fine-tuned Llama-3.2-3B on 100 held-out test examples (48 of size 2×2, 52 of size 3×3) across all three diagnostic levels described in §2.

3.1 Baseline Behavior (Untrained Model)

The untrained Llama-3.2-3B produces syntactically valid output on **100%** of test examples—every output is parseable and correctly shaped. This is itself informative: the model has sufficient prior knowledge of the serialization format to emit matrix-like text without any fine-tuning.

However, syntactic success does not translate to mathematical success. Residuals are:

$$\|AX - I\|_F = 1.835 \quad \text{and} \quad \|XA - I\|_F = 1.851 \quad \text{(avg. over 100 examples)}$$

Both values are far above any acceptable tolerance, and numeric accuracy is 0% under strict, medium, and loose thresholds alike. All 100 outputs are classified `NUMERICALLY_WRONG`.

The extended diagnostics reveal further structure:

- **Determinant ratio**: 0.045 — the determinants of predicted matrices are roughly $1/22^{\text{nd}}$ the scale of the true inverse's determinant. The model has no control over the determinant of its output.
- **Sign agreement**: 0.503 — essentially at chance, meaning the model's predictions are no more sign-aligned with the true inverse than a random matrix.
- **Relative Frobenius error**: 1.030 — predictions are as far from the true inverse, proportionally, as the inverse itself is from the origin. This confirms near-random numeric content.
- **Spectral error**: 5.015 — the dominant direction of error is large, indicating no directional bias toward the true inverse.
- **Copy distances**: $\|X - A\|_F = 1.448$, $\|X - A^T\|_F = 1.585$, $\|X - (-A)\|_F = 2.119$. The relatively small $\|X - A\|_F$ compared to $\|X - (-A)\|_F$ suggests the baseline outputs may be loosely similar in magnitude to the input, though not as a simple copy.

Summary: The baseline model is a format-competent but numerically uninformed predictor. It produces the correct output *structure* with no apparent relationship between input and output values.

3.2 Fine-Tuned Model Behavior

After LoRA fine-tuning (800 steps on 800 training examples), we observe two simultaneous effects: improvement in coarse numeric diagnostics, and degradation in generation reliability.

Generation Reliability

Only **60%** of fine-tuned outputs are parseable and correctly shaped (vs. 100% baseline). The 40-percentage-point drop is distributed across two decoding pathologies:

- `REPEATED_BLOCK_LOOP`: 22 examples (22%) — the model enters a repetition loop, exhausting the token budget without completing a valid matrix.
- `SHAPE_FORMAT`: 18 examples (18%) — the model produces partial output with wrong dimensions.

Notably, the fine-tuned model hits the 4,096-token limit on *every* example, compared to the baseline's typical 52–55 tokens per output. This suggests fine-tuning disrupted the model's length-control mechanism, causing over-generation rather than clean termination.

By matrix size: parse success is **70.8%** for 2×2 and **50.0%** for 3×3, indicating that larger matrices suffer more severe generation instability.

Numeric Diagnostics (among 60 parseable outputs)

Despite the reliability drop, the fine-tuned model's parseable outputs exhibit meaningfully improved inverse-like structure:

- **Determinant ratio**: 0.639 (vs. 0.045 baseline, $\Delta = +0.594$) — the model now produces outputs with determinants much closer to the correct scale, a substantial improvement.
- **Sign agreement**: 0.795 (vs. 0.503 baseline, $\Delta = +0.292$) — 79.5% of predicted entries have the correct sign, well above chance. The model is learning which entries should be positive or negative.
- **Relative Frobenius error**: 0.727 (vs. 1.030 baseline, $\Delta = -0.303$) — predictions are now roughly 27% closer, proportionally, to the true inverse than the baseline.
- **Spectral error**: 3.829 (vs. 5.015 baseline, $\Delta = -1.186$) — the dominant error direction has shrunk by 24%.
- **Residuals**: $\|AX - I\|_F = 1.719$ (vs. 1.835 baseline), $\|XA - I\|_F = 1.480$ (vs. 1.851 baseline). The right residual improves more substantially.
- **Copy distances**: $\|X - A\|_F = 3.175$ (vs. 1.448 baseline, $\Delta = +1.727$) — fine-tuned outputs are much farther from being a copy of the input. Similarly, $\|X - A^T\|_F$ and $\|X - (-A)\|_F$ both increase. This indicates the model has stopped producing input-like outputs and is generating genuinely different values, though those values still do not satisfy $AX = I$.

True Inverse Correctness

Despite these improvements, inverse accuracy remains **0%** under all tolerance levels (strict, medium, and loose). No parseable output satisfies $\|AX - I\|_F < 10^{-1}$.

3.3 Comparative Summary and Failure Mode Analysis

The complete comparison is shown in Table 3. Table 4 breaks down the fine-tuned failure mode distribution.

The net picture from the deltas is:

- Fine-tuning substantially reduced trivial copying behavior ($\|X - A\|_F$ rose from 1.45 to 3.18), consistent with the model learning that the inverse is *different from* the input.
- Fine-tuning improved the model's representation of what an inverse *looks like* at the level of sign, scale, and relative magnitude.
- But these improvements are insufficient to satisfy $AX \approx I$, meaning the learned structure is still too coarse or too inaccurate to yield valid inverses.
- And fine-tuning introduced a severe generation reliability penalty: a 40-point drop in parse rate, driven by looping and shape errors, that makes the fine-tuned model less deployable than the baseline at the output level.

3.4 The Three-Level Gap

The results illustrate a three-level gap between syntactic, structural, and functional success:

Level	**Baseline**	**Fine-Tuned**
Syntactic (parse + shape)	100.0%	60.0%
Structural (sign agreement)	50.3%	79.5%
Functional (any tolerance pass)	0.0%	0.0%

Fine-tuning closes the structural gap substantially but leaves the functional gap intact. The bottlenecks are insufficient numeric precision and, critically, a decoding instability that eliminates 40% of outputs before they can even be evaluated.

4 Discussion

4.1 What the Results Reveal About LLM Arithmetic Reasoning

Our findings contribute to a growing body of evidence on LLM limitations in precise arithmetic reasoning, but with a more nuanced picture than a simple pass/fail verdict.

The baseline result—100% syntactic success, 0% functional success—confirms that LLMs can learn output *format* from pretraining without any task-specific training. The model emits well-formed inverse-shaped text because it has seen similar patterns, not because it has computed anything.

The fine-tuned result is more revealing. The model did *not* simply learn the same format more reliably. Instead, it learned something genuinely new about inverse-like numeric structure:

- The 14× improvement in determinant ratio ($0.045 \rightarrow 0.639$) indicates the model learned to modulate its output's scale relative to the input's scale, consistent with implicit knowledge that the inverse's determinant is the reciprocal of the input's determinant.
- The 29-point gain in sign agreement ($0.503 \rightarrow 0.795$) indicates the model is learning which entries of the true inverse are positive versus negative—a genuine input-output dependency, since sign patterns depend on the specific values of A.
- The 30-point reduction in relative Frobenius error indicates predictions closer to the true inverse, not just closer to a plausible format.

- The increase in copy distances ($\|X - A\|_F$ grew from 1.45 to 3.18) confirms the model is not simply echoing the input; it is generating substantially different values.

Together, these metrics indicate that supervised fine-tuning did induce some dependency between input and output at the level of coarse numeric structure. The model appears to have learned *partial inverse-like patterns*: sign, approximate scale, and general numeric character, but not the precise multi-step algebraic constraint $AX = I$.

4.2 Why the Functional Gap Persists

Despite measurable structural improvement, no output satisfies $\|AX - I\|_F < 10^{-1}$. Several factors likely contribute:

1. **Insufficient training signal**: 800 examples and 800 steps may be insufficient for a 3-billion-parameter model to generalize a precise function from noisy weight updates. Matrix inversion requires entrywise precision across all 4 or 9 output values; a single misestimated entry can cause large residuals.

2. **Representation mismatch**: LLMs operate on token sequences, not numerical values. Each floating-point number is tokenized as a sequence of characters without any intrinsic numeric semantics. The model must implicitly learn to "parse" input tokens as numbers, compute interactions, and "render" output tokens—all within a continuous embedding space that was not designed for this purpose.

3. **Lack of intermediate computation**: Matrix inversion by Gaussian elimination requires structured intermediate steps. Supervised fine-tuning on final (A, A^{-1}) pairs provides no signal about these intermediate steps, forcing the model to learn a direct mapping that may exceed what a small model can approximate.

4. **Decoding instability as a bottleneck**: The 40% parse failure rate means even evaluable outputs are a *selected* subset—likely those where the model happened to produce shorter, more structured outputs. The improvement in diagnostics among parseable examples may be partly a selection artifact. It is possible the true distribution of fine-tuned outputs is bimodal: some are structurally improving, while others have entirely collapsed into repetition loops.

4.3 Decoding Instability as a First-Class Finding

The generation reliability drop—from 100% to 60% parseable—is not a peripheral implementation detail. It is a central result. Fine-tuning caused the model to lose its ability to produce clean, length-controlled outputs on a majority of 3×3 examples, with every output hitting the 4,096-token budget.

This suggests that the LoRA updates that improved numeric structure also destabilized the model's autoregressive termination behavior. The model may be attempting more "computation" in its output but getting caught in degenerate loops when that computation doesn't converge to a clean termination state.

From a systems perspective, a model that is more numerically aware but less generatively reliable is not obviously better than the baseline. The right metric for deployment is not just numeric accuracy on parseable outputs, but accuracy on *all* outputs. By that measure, the fine-tuned model is worse: 0% success vs. 0% success with significantly more wasted computation.

4.4 Implications for the Broader Question

Our experiment probes a specific version of a general question: can LLMs learn to compute functions that humans find computationally demanding, purely from (input, output) example pairs? For matrix inversion, the answer at this scale is *not yet*. But the diagnostic structure of the failure provides direction:

- The model is not stuck at zero—it is learning something. The path forward likely involves more training data, more steps, or structural changes to the supervision signal.
- Format and reliability are prerequisites for numeric learning. The decoding instability must be resolved before further numeric gains can be realized.
- The results are consistent with a hypothesis of *capacity limitation*, not a fundamental impossibility: a sufficiently large model with sufficiently structured supervision might succeed where this small model fails.

4.5 Limitations

1. **Single model and architecture**: Llama-3.2-3B with LoRA. Larger models or different adapter configurations may behave differently.
2. **No hyperparameter search**: We use one configuration. Different ranks, learning rates, or training lengths may substantially change outcomes.
3. **Fixed output format**: The serialization format may be suboptimal. A simpler format (e.g., space-separated values without headers) might reduce formatting failures.
4. **Parser brittleness**: Some outputs classified as `SHAPE_FORMAT` or `REPEATED_BLOCK_LOOP` may contain valid partial matrices; our parser may be more conservative than necessary. True numeric performance may be slightly underestimated.
5. **Small matrix scale**: 3×3 is a restricted domain. Behavior on larger matrices, where the inverse is harder to approximate, is unknown.
6. **No intermediate supervision**: We provide only final (A, A^{-1}) pairs. Chain-of-thought or step-by-step computation supervision might substantially change the model's learning trajectory.

5 Conclusion

We investigated whether small language models can learn matrix inversion through supervised LoRA fine-tuning on synthetic (A, A^{-1}) pairs—a controlled test of LLM capability on a computation that is non-trivial for humans yet verifiable with a simple residual check. Our three-level diagnostic framework reveals a nuanced picture.

What succeeded: Fine-tuning produced measurable improvements across multiple coarse numeric diagnostics. The determinant ratio improved from 0.045 to 0.639 (+0.594), sign agreement from 0.503 to 0.795 (+0.292), and relative Frobenius error to the gold inverse fell from 1.030 to 0.727 (−0.303). The fine-tuned model also moved its outputs substantially farther from trivial input copying ($\|X - A\|_F$: 1.448 → 3.175). These gains indicate that the model internalized *partial inverse-like numeric structure* from the training pairs—it learned something about what inverses look like, not merely how to format them.

What failed: True inverse accuracy remained 0% under all residual-based tolerances for both models. The fine-tuned model could not bridge the gap from structural plausibility to

functional correctness. Critically, fine-tuning also introduced severe generation instability: only 60% of fine-tuned outputs were parseable (vs. 100% baseline), with 22% failing via repetition loops and 18% via shape malformations.

The dominant bottleneck: The experiment reveals two compounding failure modes. The first is *insufficient numeric precision*—the model learns coarse structure but cannot produce entrywise-accurate values. The second is *generation instability*—fine-tuning disrupted length control, causing degenerate looping on the majority of 3×3 examples. Resolving the reliability bottleneck is a prerequisite before further numeric progress can be meaningfully assessed.

Outlook: This result is a diagnostic, not a verdict. The model is not learning nothing—it is learning partial structure. Future work should prioritize: (1) simplifying the output serialization format to reduce format-induced failures; (2) decoding stabilization via repetition penalties or constrained generation; (3) tiny-set overfitting experiments to separate model capacity limits from format brittleness; and (4) intermediate-step supervision that gives the model signal about the computation process, not just the final answer. Only after reliability is established does it become meaningful to scale to larger matrices, different models, or expanded hyperparameter search.

This work contributes a careful negative-result baseline: it establishes what a 3-billion-parameter model with minimal LoRA tuning can and cannot achieve on a precise arithmetic task, with a diagnostic framework that isolates syntactic, structural, and functional failure independently.

References

References

[1] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training verifiers to solve math word problems*. arXiv preprint arXiv:2110.14168.

[2] Hu, E. Q., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv preprint arXiv:2106.09685.

[3] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., & others (2023). *Llama 2: Open foundation and fine-tuned chat models*. arXiv preprint arXiv:2307.09288.

A Tables

Table 1: Stability Criteria and Dataset Generation Statistics

Parameter	**Value**
Matrix sizes	2×2 and 3×3
Samples per size (target)	500
Total dataset size	1,000 examples
Train / Val / Test split	800 / 100 / 100
Test set composition	48 of size 2×2; 52 of size 3×3
Random distribution	Uniform $[-1, 1]$
Determinant threshold	$\lvert\det(A)\rvert > 0.05$
Condition number bound	$\kappa(A) < 50$
Residual tolerance (gold)	$\lVert AX - I\rVert_F < 10^{-8}$
Max generation attempts	20,000 per size
Decimal precision (serialization)	5
Split seed	123

Table 2: Training Configuration and Hyperparameters

Component	**Configuration**
Base model	Llama-3.2-3B
Adapter method	LoRA (Low-Rank Adaptation)
LoRA rank (r)	16
LoRA alpha (α)	32
Effective scaling (α/r)	2.0
Dropout	0.05
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
Learning rate	5×10^{-5}
Weight decay	0.01
Warmup ratio	5%
Training steps	800
Batch size	16
Training examples	800 (all from train split)
Max sequence length	2,048 tokens
Inference decoding	Greedy (temperature = 0.0)
Max new tokens	128
Training infrastructure	Tinker SDK (async pipelining)

Table 3: Full Diagnostic Comparison: Baseline vs. Fine-Tuned on Test Set ($n = 100$ examples; 48 of size 2×2, 52 of size 3×3). Extended metrics are computed over parseable outputs only (100 baseline, 60 fine-tuned). $\uparrow$ = higher is better; $\downarrow$ = lower is better.

Metric	**Baseline**	**Fine-Tuned**	**Delta**
Generation Reliability			
Parse / shape rate	100.00%	60.00%	−40.00 pp
True Inverse Correctness (Level 3)			
Accuracy — strict ($\|AX - I\|_F < 10^{-2}$)	0.0%	0.0%	+0.0 pp
Accuracy — medium ($< 5 \times 10^{-2}$)	0.0%	0.0%	+0.0 pp
Accuracy — loose ($< 10^{-1}$)	0.0%	0.0%	+0.0 pp
Avg $\|AX - I\|_F \downarrow$	1.8354	1.7191	−0.1163
Avg $\|XA - I\|_F \downarrow$	1.8513	1.4800	−0.3713
Coarse Numeric Structure (Level 2, parseable only)			
Avg relative $\|X - A^{-1}\|_F \downarrow$	1.0301	0.7273	−0.3028
Avg determinant ratio $\uparrow$	0.0451	0.6390	+0.5939
Avg sign agreement $\uparrow$	0.5033	0.7949	+0.2916
Avg spectral error $\downarrow$	5.0149	3.8292	−1.1857
Copy-Detection Distances (parseable only)			
Avg $\|X - A\|_F$ (input copy?)	1.4476	3.1753	+1.7277
Avg $\|X - A^T\|_F$ (transpose copy?)	1.5852	2.7599	+1.1747
Avg $\|X - (-A)\|_F$ (negation copy?)	2.1186	3.9595	+1.8410

Table 4: Fine-Tuned Model Failure Mode Breakdown ($n = 100$ test examples). The baseline has no looping or shape failures; all 100 baseline outputs are `NUMERICALLY_WRONG`.

Failure Mode	**Count**	**Percent**
`NUMERICALLY_WRONG`	60	60.0%
`REPEATED_BLOCK_LOOP`	22	22.0%
`SHAPE_FORMAT`	18	18.0%
`SUCCESS`	0	0.0%
Total	100	100.0%

B Figures

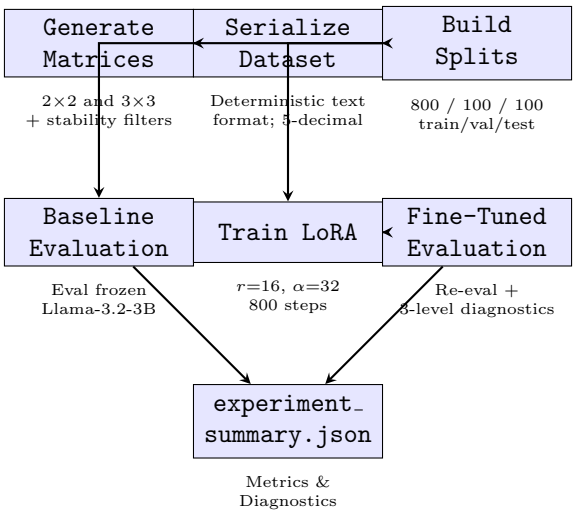


Figure 1: Six-stage MatrixInvLLM pipeline. (1) Generate stable 2×2 and 3×3 matrices with determinant ($|\det A| > 0.05$) and condition number ($\kappa < 50$) filters; 500 examples per size. (2) Serialize to deterministic text format with 5-decimal precision. (3) Split 1,000 examples into 800 train / 100 val / 100 test. (4) Evaluate untrained Llama-3.2-3B baseline on 100 test examples. (5) Apply LoRA fine-tuning (r=16, α=32, 800 steps) on training set. (6) Re-evaluate fine-tuned model using a three-level diagnostic suite: syntactic well-formedness, coarse numeric structure, and residual-based inverse correctness.

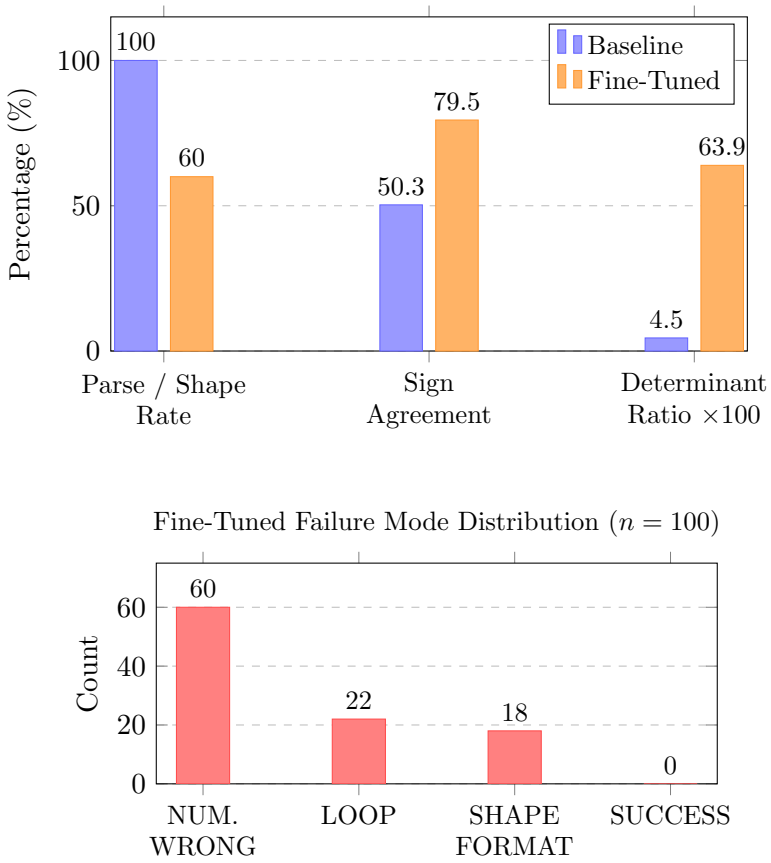


Figure 2: **Top**: Three normalized metrics (all higher = better) comparing baseline and fine-tuned model. Parse/shape rate measures generation reliability; sign agreement and determinant ratio (×100 for scale) measure inverse-like coarse structure among parseable outputs. Fine-tuning improves both structural metrics—sign agreement rises from 50.3% to 79.5%, determinant ratio from 4.5% to 63.9%—but reduces parse reliability by 40 percentage points. Additional diagnostics (residuals, relative Frobenius error, spectral error, copy distances) are in Table 3. **Bottom**: Failure mode distribution for the fine-tuned model ($n = 100$ test examples). 60 outputs are parseable but numerically wrong; 22 loop until the token budget is exhausted; 18 produce malformed shapes; 0 achieve a successful inversion.